

JAVASCRIPT COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ *Red star = Important Topic (read carefully, exam / interview favorite)*

MMOC TEACH
MADE BY PRAVENDRA

INDEX

- ★ 1. Introduction to JavaScript
- ★ 2. Variables - var, let, const
- 3. Data Types
- 4. Operators
- ★ 5. Conditional Statements (if, else if, else)
- ★ 6. Loops - for, while
- ★ 7. Functions
- 8. Arrow Functions
- ★ 9. Arrays and Array Methods
- ★ 10. Objects
- 11. String Methods
- ★ 12. DOM Manipulation
- 13. Events
- ★ 14. Promises and Async/Await
- 15. JSON

★ 1. Introduction to JavaScript *[IMPORTANT TOPIC]*

Explanation:

JavaScript is a programming language used to make websites interactive. While HTML builds the structure and CSS makes it look nice, JavaScript is what makes a webpage actually DO things — like showing a pop-up, validating a form, or updating content without reloading the page.

JavaScript runs directly inside the web browser, so no separate installation is needed to run it on a website.

Key Points:

- Runs in every web browser (Chrome, Firefox, etc.).
- Used for both front-end (browser) and back-end (using Node.js) development.
- Case-sensitive language — 'Name' and 'name' are treated as different.

Example Code:

```
console.log("Hello, World!");
```

Output:

```
Hello, World!
```

💡 *Note: console.log() is used to print output — usually visible in the browser's Developer Console (press F12).*

★ 2. Variables - var, let, const [IMPORTANT TOPIC]

Explanation:

A variable stores a value we can use later. JavaScript has three ways to create a variable: 'var' (old style), 'let' (modern, value can change), and 'const' (modern, value CANNOT change after being set).

Example Code:

```
let name = "Pravendra";  
let age = 21;  
const country = "India";  
  
age = 22;    // allowed, since 'let' can change  
console.log(name, age, country);
```

Output:

Pravendra 22 India

💡 *Note: Always prefer 'let' and 'const' over 'var' in modern JavaScript — they are safer and avoid common bugs.*

3. Data Types

Explanation:

JavaScript has several basic data types to represent different kinds of values.

Key Points:

- Number -> 10, 3.14
- String -> "hello"
- Boolean -> true / false
- Undefined -> a variable declared but not yet given a value
- Null -> represents 'no value' on purpose
- Object / Array -> to store multiple or complex values

Example Code:

```
let a = 10;  
let b = "text";  
let c = true;  
console.log(typeof a, typeof b, typeof c);
```

Output:

```
number string boolean
```

4. Operators

Explanation:

Operators perform actions on values, such as maths, comparisons, or logic.

Key Points:

- Arithmetic: + - * / %
- Comparison: == === != !== > <
- Logical: && || !

Example Code:

```
console.log(5 + 3);           // 8
console.log(5 === "5");      // false (checks type too)
console.log(5 == "5");       // true (does not check type)
```

Output:

```
8
false
true
```

 *Note: Always prefer === (strict equality) over == — it avoids confusing type-conversion bugs.*

★ 5. Conditional Statements (if, else if, else) *[IMPORTANT TOPIC]*

Explanation:

Conditional statements let the program make decisions. Code inside 'if' runs only when the condition is true.

Example Code:

```
let marks = 75;

if (marks >= 90) {
  console.log("Grade A");
} else if (marks >= 60) {
  console.log("Grade B");
} else {
  console.log("Grade C");
}
```

Output:

Grade B

★ 6. Loops - for, while *[IMPORTANT TOPIC]*

Explanation:

Loops repeat a block of code multiple times, instead of writing it again and again.

Example Code:

```
for (let i = 1; i <= 3; i++) {  
    console.log("Count: " + i);  
}
```

Output:

```
Count: 1  
Count: 2  
Count: 3
```

★ 7. Functions *[IMPORTANT TOPIC]*

Explanation:

A function is a reusable block of code that performs a specific task. We write it once and 'call' it whenever we need it.

Example Code:

```
function addNumbers(a, b) {  
    return a + b;  
}  
  
let total = addNumbers(5, 7);  
console.log("Total:", total);
```

Output:

Total: 12

8. Arrow Functions

Explanation:

Arrow functions are a shorter, modern way to write functions in JavaScript, using the => symbol.

Example Code:

```
const add = (a, b) => {  
  return a + b;  
};  
  
console.log(add(4, 6));  
  
// even shorter for single-line functions  
const square = x => x * x;  
console.log(square(5));
```

Output:

```
10  
25
```

★ 9. Arrays and Array Methods *[IMPORTANT TOPIC]*

Explanation:

An array stores multiple values together in one variable, in order. JavaScript arrays come with many useful built-in methods.

Key Points:

- `push()` -> add item at the end
- `pop()` -> remove the last item
- `map()` -> create a new array by transforming each item
- `filter()` -> create a new array with only matching items

Example Code:

```
let fruits = ["apple", "banana", "mango"];
fruits.push("orange");
console.log(fruits);

let doubled = [1, 2, 3].map(n => n * 2);
console.log(doubled);
```

Output:

```
[ 'apple', 'banana', 'mango', 'orange' ]
[ 2, 4, 6 ]
```

★ 10. Objects *[IMPORTANT TOPIC]*

Explanation:

An object stores data as 'key : value' pairs, similar to a real-world record. Objects are used to group related information together.

Example Code:

```
let student = {  
  name: "Pravendra",  
  age: 21,  
};  
  
console.log(student.name);  
student.age = 22;  
console.log(student);
```

Output:

```
Pravendra  
{ name: 'Pravendra', age: 22 }
```

11. String Methods

Explanation:

JavaScript strings come with many built-in methods to change or check text easily.

Example Code:

```
let text = "  Hello JS  ";  
console.log(text.trim());  
console.log(text.trim().toUpperCase());  
console.log(text.trim().length);
```

Output:

```
Hello JS  
HELLO JS  
8
```

★ 12. DOM Manipulation *[IMPORTANT TOPIC]*

Explanation:

The DOM (Document Object Model) is how JavaScript sees and controls a webpage's content. Using the DOM, JavaScript can change text, styles, or structure of a page WITHOUT reloading it.

Example Code:

```
// Changes the text of an element with id='title'
document.getElementById("title").innerText = "Welcome!";

// Changes its background color
document.getElementById("title").style.color = "blue";
```

Output:

Screen shows: the element with id 'title' now displays 'Welcome!' in blue text.

13. Events

Explanation:

An event is an action that happens on a webpage, like a click, a key press, or moving the mouse. JavaScript can 'listen' for these events and run code in response.

Example Code:

```
document.getElementById("myButton").addEventListener("click", function() {  
    alert("Button was clicked!");  
});
```

Output:

Screen shows: a popup box saying 'Button was clicked!' whenever the button is pressed.

★ 14. Promises and Async/Await *[IMPORTANT TOPIC]*

Explanation:

A Promise represents a task that will finish LATER, like fetching data from the internet. async/await is a cleaner, modern way to work with Promises, making the code look like it runs step-by-step even though it's waiting for something.

Example Code:

```
async function getData() {  
  let response = await fetch("https://api.example.com/data");  
  let data = await response.json();  
  console.log(data);  
}
```

Output:

(Once the internet request finishes, the fetched data is printed.)

 *Note: 'await' can only be used inside a function marked 'async'.*

15. JSON

Explanation:

JSON (JavaScript Object Notation) is a simple text format used to store and exchange data, especially between a website and a server. It looks very similar to a JavaScript object.

Example Code:

```
let student = { name: "Pravendra", age: 21 };

let jsonText = JSON.stringify(student);    // object -> JSON text
console.log(jsonText);

let backToObject = JSON.parse(jsonText);  // JSON text -> object
console.log(backToObject.name);
```

Output:

```
{"name":"Pravendra","age":21}
Pravendra
```